FQL-X: Keeping All Offline Data and Updating More Often for Offline-to-Online Flow Q-Learning

CS 185/285 Final Project (Offline-to-Online RL), Spring 2026

Abstract

A policy that is pre-trained on offline data should improve when it is allowed to interact with the environment, but naive fine-tuning often makes it worse first. I study this problem with Flow Q-Learning (FQL) on three OGBench tasks. Naive online fine-tuning of FQL with an initially empty replay buffer collapses on the two harder tasks (final success 0.14 on cube-double and 0.00 on antsoccer-arena, mean of two seeds). I implement five remedies from the literature (retaining offline data, warm-start RL, IFQL with rejection sampling, DSRL, and Q-score matching) and compare them. My method, FQL-X, makes three changes to FQL's online phase and none to its losses: the replay buffer is initialised with the entire offline dataset, four gradient updates are made per environment step, and actions can be selected by a pessimistic best-of-N rule (which the experiments show should be switched off). After 500K offline and 100K online steps, FQL-X reaches 0.96 on cube-double and 0.88 on antsoccer-arena (mean of two seeds), above the project targets of 0.95 and 0.45 and above all six baselines. The gain over the strongest baseline (FQL with retained offline data) is small in the final evaluation and larger in the early online phase.

1. Introduction

Offline RL learns a policy from a fixed dataset; online RL then continues from this policy with real interaction. The transition is difficult for two reasons. First, the critic was trained only on dataset actions and is unreliable elsewhere, so the first online updates can follow wrong value estimates. Second, the training distribution changes abruptly: a freshly created replay buffer contains a few thousand transitions from one policy, while the networks were fitted to a million diverse transitions.

All experiments use the same protocol: 500K offline gradient steps, then 100K online environment steps, evaluation on 25 episodes every 50K steps offline and every 20K steps online. One evaluation therefore has a resolution of 0.04 and a standard error of up to 0.10, and differences of that size between two runs are noise. The tasks are `cube-single-play` (sanity check), `cube-double-play` and `antsoccer-arena-navigate` (all `singletask-task1-v0`). Rewards are −1 per step until success, so Q-values lie in about $[-100, 0]$ with a discount of 0.99.

The report first shows the failure of naive fine-tuning (Section 4.1), then evaluates the five remedies (Section 4.2), and finally the proposed method with its ablations (Section 4.3).

2. Related work and baselines

SAC+BC trains a Gaussian actor to maximise Q plus a behaviour-cloning (BC) term weighted by α. **FQL** (Park et al., 2025) trains a flow-matching BC policy and distils it into a one-step policy that also maximises Q; α weights the distillation term. FQL is the better base on the harder tasks in Assignment 5, so all later baselines build on it.

Retaining offline data (as in RLPD; Ball et al., 2023) pre-fills the online replay buffer with offline transitions. **WSRL** (Zhou et al., 2025) keeps no offline data but first collects a number of warm-up transitions with the frozen pre-trained policy before any update. **IFQL** uses implicit Q-learning for the critic, a flow BC policy as the actor, and best-of-N (rejection) sampling of actions with the critic. **DSRL** (Wagenmaker et al., 2025) keeps a flow BC policy and trains a second policy that chooses the *noise* fed into it. **QSM** (Psenka et al., 2025) trains a diffusion policy whose score is matched to the action gradient of Q, with an additional DDPM (BC) term as in the QAM baseline (Li and Levine, 2026).

FQL-X combines the data retention of RLPD with a higher update-to-data (UTD) ratio, also from RLPD, on top of FQL's behaviour-regularised actor, and it tests whether the best-of-N action selection of IFQL helps as an add-on.

3. Method: FQL-X

FQL-X keeps the FQL losses unchanged. With critics $Q_{\theta_1}, Q_{\theta_2}$, BC flow v_ϕ and one-step policy $\mu_\omega(s, z)$:

$$L(Q) = \sum_i \mathbb{E}\left[\left(Q_{\theta_i}(s,a) - r - \gamma(1-d)\,\frac{1}{2}\sum_j \bar{Q}_{\theta_j}(s', \mu_\omega(s',z))\right)^2 \right]$$

$$L(\mu) = \mathbb{E}\left[-\frac{1}{2}\sum_j Q_{\theta_j}(s, \mu_\omega(s,z)) + \alpha \left\| \mu_\omega(s,z) - a_\phi^{\text{flow}}(s,z) \right\|^2 \right]$$

The changes concern only the online phase.

1. **The whole offline dataset stays in the replay buffer.** The buffer is initialised with all 1M offline transitions and online transitions are appended; minibatches are sampled uniformly. The critic therefore keeps training on the distribution it was fitted to, and the BC flow keeps its broad action coverage. The fraction of online data grows slowly (to 9% after 100K steps), so the training distribution changes gradually instead of abruptly.
2. **Update-to-data ratio 4.** Four gradient updates (critic, BC flow and one-step policy) are made per environment step. With the offline data in the buffer, the extra updates do not over-fit a small online buffer, and each new transition, in particular each new success, is propagated through the critic sooner. Online interaction is the limited resource (100K steps), so more computation per step is the natural trade.
3. **Optional pessimistic best-of-N action selection.** For both exploration and evaluation, N candidates $a_j = \mu_\omega(s, z_j)$ are drawn and the one with the largest $\text{mean}_i Q_{\theta_i}(s,a_j) - \rho\,\text{std}_i Q_{\theta_i}(s,a_j)$ is executed. $N = 1$ gives FQL's normal action. The ablations below show that $N > 1$ is harmful on antsoccer-arena, so the final method uses $N = 1$.

Two further options were implemented and tested: a different α for the online phase (`online_alpha`), and symmetric sampling, in which a fixed fraction of every minibatch comes from the offline dataset and the rest from an online-only buffer (`--offline_ratio`). Neither is used in the final method.

Final configuration. FQL hyperparameters from Assignment 5 (α = 300 on the cube tasks, α = 10 on antsoccer-arena; 4 × 512 MLPs, learning rate 3e-4, batch size 256, 10 flow steps), offline data 1,000,000, UTD 4, $N = 1$. The offline phase is identical to FQL, so FQL-X starts from the same offline checkpoint as the FQL baselines of the same seed. The implementation is `src/agents/fqlx_agent.py`, `src/configs/fqlx_config.py`, and the options `--offline_data`, `--utd`, `--offline_ratio` in `src/scripts/train_offline_online.py`.

4. Experiments

4.1 Naive offline-to-online baselines (Part I)

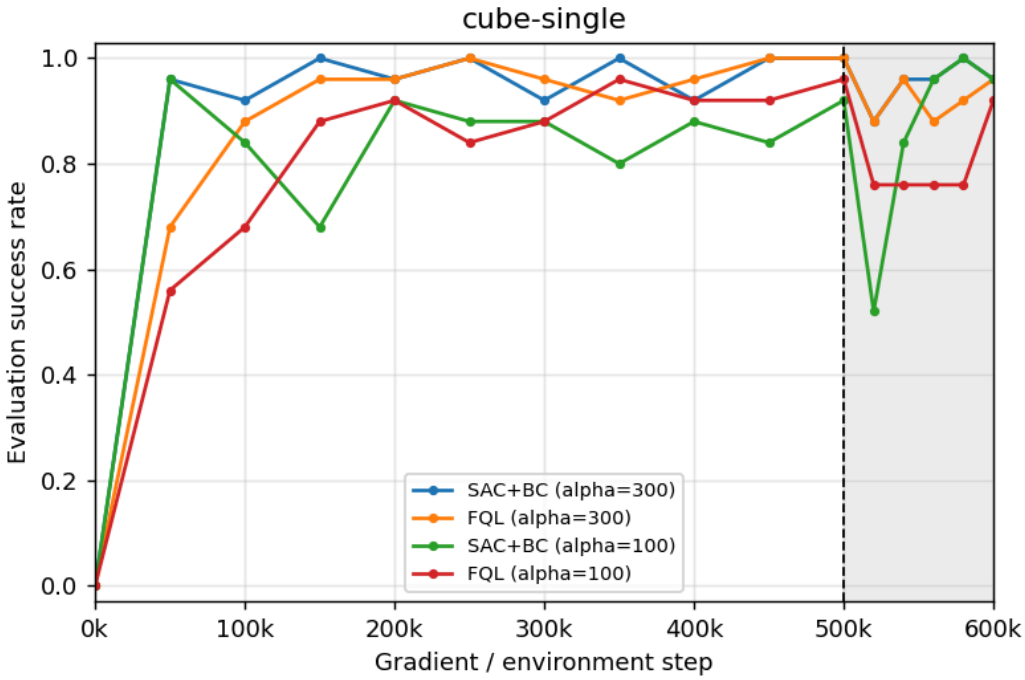


Figure 1: SAC+BC and FQL on cube-single. The dashed line and grey area mark the online phase.

Both methods reach the offline target on cube-single (1.00 at 500K steps with $\alpha = 300$). When online training starts with an empty replay buffer, the success rate dips at the first online evaluation (to 0.88 for both with $\alpha = 300$, and to 0.52 for SAC+BC and 0.76 for FQL with $\alpha = 100$) and then recovers to 0.92–0.96 at 600K steps.

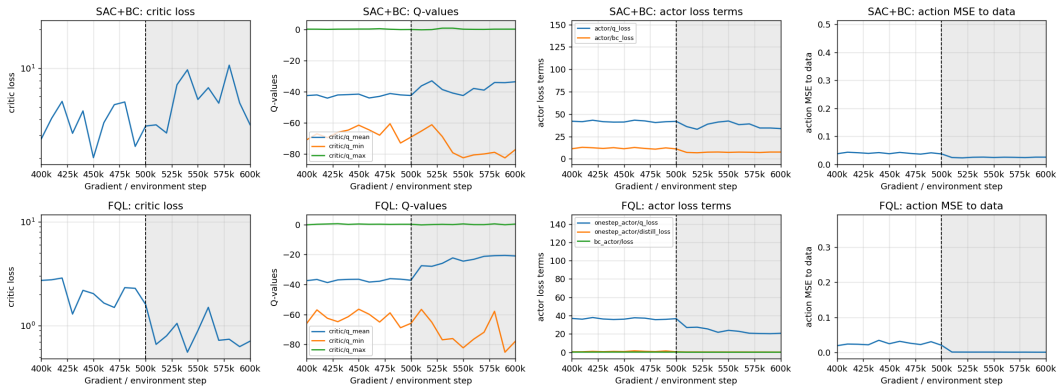


Figure 2: Training metrics around the switch to online training (cube-single, $\alpha = 300$).

Question 1: what do the Q-values, losses and action MSE show? At the switch, the minibatches change from the diverse offline data to the agent's own recent transitions. Three effects are visible. (i) The action MSE between policy and buffer actions drops at once (FQL: from about 0.02 to below 0.002) because the buffer actions are now the policy's own actions; the BC term therefore stops constraining the policy to the *offline* data and only ties it to itself. (ii) The mean Q-value rises (FQL: from −37 to −21, SAC+BC: from −42 to −34), because the online buffer of a mostly successful policy contains mainly short, successful trajectories. (iii) The critic loss of SAC+BC becomes larger and noisier (from about 3 to 5–10), while that of FQL falls, since the narrow online data is easy to fit. On cube-single the pre-trained policy already succeeds, so its own data is good data and the process is stable after a short dip.

Question 2: why does naive fine-tuning fail on harder tasks? On cube-double and antsoccer-arena the same procedure collapses:

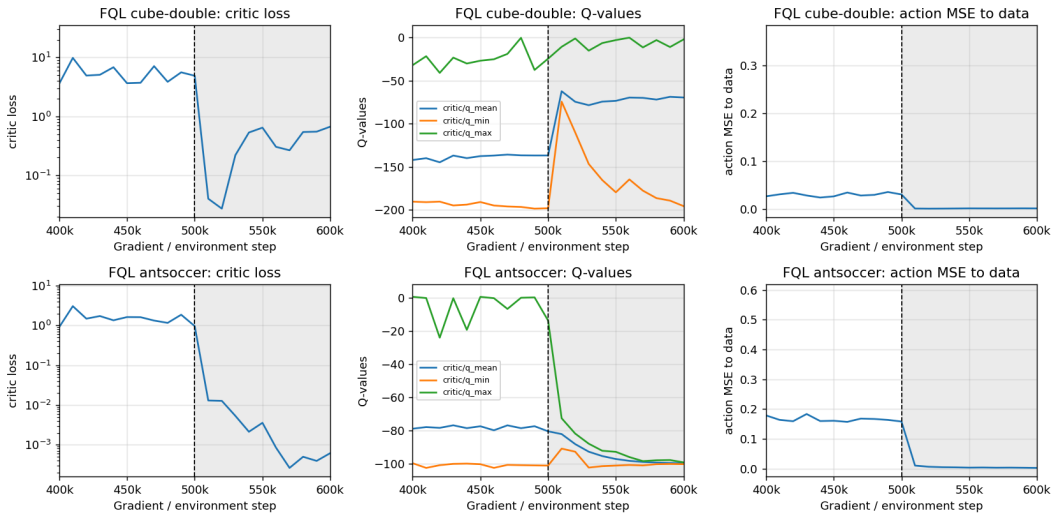


Figure 3: Naive FQL on cube-double and antsoccer-arena.

Task (naive FQL)	Offline (500K)	520K	600K
cube-double, seeds 0 / 1	0.52 / 0.56	0.28 / 0.08	0.24 / 0.04
antsoccer-arena, seeds 0 / 1	0.44 / 0.60	0.00 / 0.00	0.00 / 0.00

Here the pre-trained policy succeeds only about half of the time, so the first online buffer contains many failures and no broad coverage. On antsoccer-arena the action MSE drops from 0.16 to 0.01 within 10K steps: the BC constraint has been replaced by imitation of the agent's own recent behaviour. The critic, now trained only on this narrow data, forgets the values it had learned for other states and actions; its predictions contract to a narrow band (maximum Q from −14 to −73 after 10K steps) and converge to −100 for all inputs at 600K steps, the value of never succeeding. Once the policy has drifted and stopped succeeding, the buffer contains no success, so nothing can pull the values back: the collapse reinforces itself. The offline data that defined both the behaviour constraint and the critic's support is the missing ingredient, which motivates the methods of the next section.

4.2 Improved baselines (Part II)

Retaining offline data.

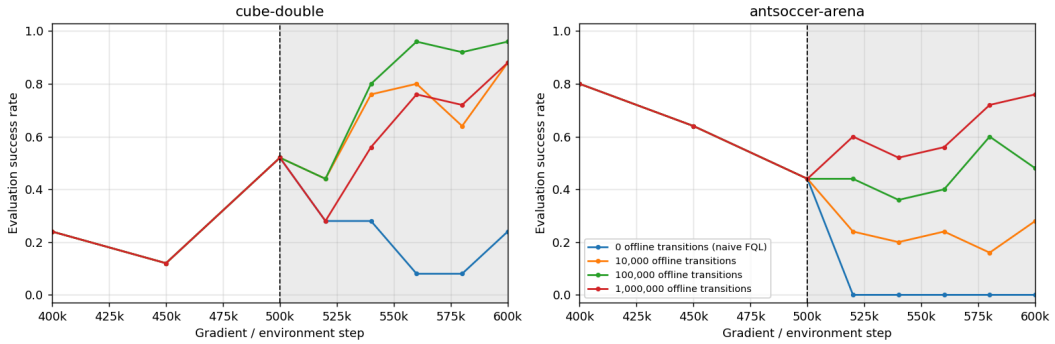


Figure 4: FQL with 0, 10K, 100K and 1M offline transitions pre-filled into the online buffer (seed 0).

Offline transitions	cube-double: 520K / 600K	antsoccer-arena: 520K / 600K
0 (naive)	0.28 / 0.24	0.00 / 0.00
10,000	0.44 / 0.88	0.24 / 0.28
100,000	0.44 / 0.96	0.44 / 0.48
1,000,000	0.28 / 0.88	0.60 / 0.76

Question 1 (data distribution). Without offline data, minibatches come only from the state–action distribution of the current policy. With retained data they come from a mixture of the offline distribution and the online one, with mixture weights proportional to the two sizes. The offline part keeps the critic accurate on the broad support it was trained on and keeps the BC term pointing at the dataset actions, while the online part adds the states the policy actually visits and corrects over-estimated values there. On cube-double every amount prevents the collapse and the three amounts are within noise of each other at the end. On antsoccer-arena more data is clearly better: 10K transitions are a small subset that covers the large maze poorly and is soon outnumbered by online data.

WSRL.

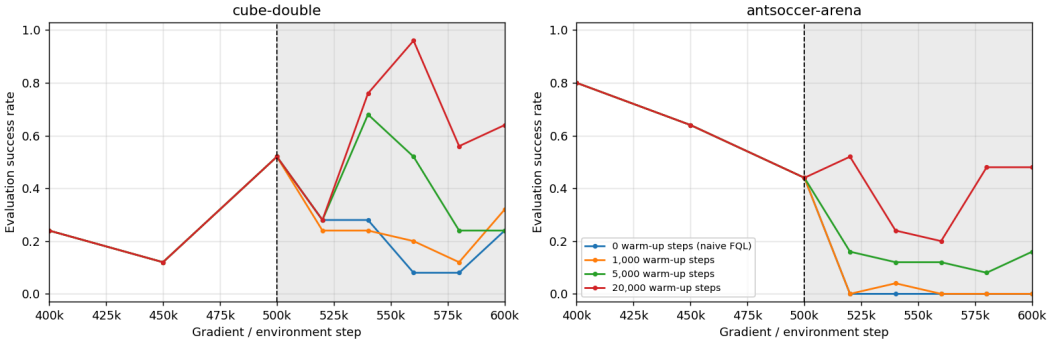


Figure 5: FQL with 0, 1K, 5K and 20K warm-up steps and no offline data (seed 0).

Warm-up steps	cube-double: 520K / 600K	antsoccer-arena: 520K / 600K
1,000	0.24 / 0.32	0.00 / 0.00
5,000	0.28 / 0.24	0.16 / 0.16
20,000	0.28 / 0.64	0.52 / 0.48

Question 2 (WSRL). With retained offline data the buffer contains the behaviour-policy data of the dataset. With WSRL it contains no offline data at all; instead it starts with transitions collected by the *frozen pre-trained policy*, followed by the data of the learning policy. This warm-up data is on-policy for the initial agent: it covers exactly the states and actions where the critic will be queried first, and its size prevents the first updates from being computed on a handful of correlated transitions. This is better than starting from an empty buffer because it removes the abrupt shift that causes the collapse, and it can be better than retaining offline data because the offline data may be far from what the policy does (sub-optimal or irrelevant behaviour), which keeps the critic pessimistic and slows adaptation; it is also the only option when the offline data is not available at fine-tuning time. In my experiments the warm-up must be long (20K steps, a fifth of the online budget) to help, and even then WSRL stays below data retention (final mean over two seeds 0.66 versus 0.88 on cube-double and 0.52 versus 0.80 on antsoccer-arena). The warm-up buffer is narrower than the offline dataset and the steps spent on it are not used for learning.

IFQL, DSRL and QSM.

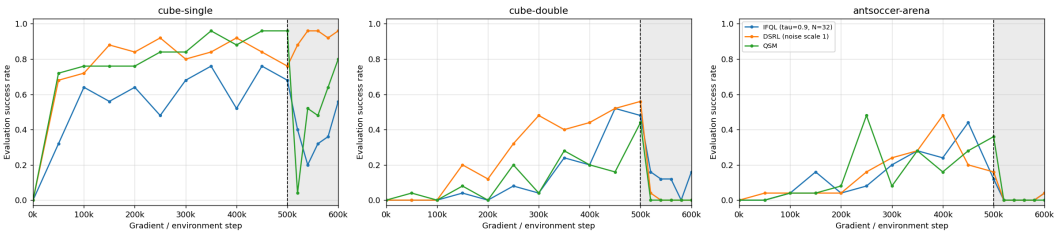


Figure 6: IFQL, DSRL and QSM (seed 0).

Method (setting)	cube-single: offline / final	cube-double: offline / final	antsoccer-arena: offline / final
IFQL (τ = 0.9, N = 32)	0.68 / 0.56	0.48 / 0.16	0.12 / 0.04
DSRL (σ_z = 1.0)	0.76 / 0.96	0.56 / 0.00	0.16 / 0.04
QSM (α = 100, η = 10; antsoccer: α = 30, η = 30, mean target, see below)	0.96 / 0.80	0.44 / 0.00	0.36 / 0.00

All three reach the offline sanity thresholds of the handout, with two qualifications: IFQL on antsoccer-arena is 0.12 at 500K steps with seed 0 (threshold 0.15; it is 0.44 at 450K and 0.20 at 500K with seed 1), and QSM on antsoccer-arena reaches its threshold only after a change of the critic target, discussed below. All three are run with the naive online protocol (empty buffer), and all three lose most of their performance online on the two harder tasks, for the reason given in Section 4.1: their critics are also trained on the narrow online buffer only.

Question 3 (rejection sampling). Best-of-N sampling draws N actions from the BC policy and executes the one with the highest Q-value. The intuition is that the BC policy proposes only in-distribution actions, and the critic then only has to *rank* a few plausible candidates instead of being maximised over the whole action space, where it is unreliable. The executed action is distributed as the maximum-Q element of N samples: the behaviour distribution re-weighted towards high Q, with density $N\,\pi_\beta(a \mid s)\,F(Q(s,a))^{N-1}$, where F is the distribution function of Q under π_β. For a uniformly random policy, the same procedure returns the best of N random actions, which approaches $\arg\max_a Q(s,a)$. As $N \to \infty$ the policy becomes the greedy policy *restricted to the support of the behaviour policy*, $\arg\max_{a \in \operatorname{supp} \pi_\beta(\cdot \mid s)} Q(s,a)$, which is the policy that in-sample methods such as IQL aim at. N controls the trade-off between improvement and the risk of selecting an action whose value is over-estimated.

Question 4 (DSRL). With a fixed BC policy, DSRL only trains a small policy over the latent noise. (i) *Efficiency:* the RL problem is small and well-conditioned; no gradient has to be propagated through the multi-step denoising chain, and the expensive diffusion policy is not retrained. (ii) *Exploration:* any latent noise is mapped by the BC policy to an action that looks like dataset behaviour, so exploration happens on the data manifold and is structured and safe rather than random jitter in action space. (iii) *Access:* the method only needs to call the BC policy with a chosen noise input. It needs neither the weights nor gradients, so it also works with a large black-box policy behind an API. In my implementation (which, as in QAM, also keeps training the BC policy) DSRL is the best naive method on cube-single (0.96 after online training), but on the harder tasks its critic collapses like the others.

Question 5 (QSM). If $\pi(a \mid s) \propto e^{\eta Q(s,a)}$, then $\nabla_a \log \pi(a \mid s) = \eta \nabla_a Q(s,a)$, because the normalising constant does not depend on a. A diffusion model represents a distribution through its score, and its noise prediction is the score up to a negative scale, $\epsilon_\phi(s, a_t, t) \approx -\sqrt{1-\hat{\alpha}_t}\,\nabla_{a_t} \log p_t(a_t \mid s)$. Regressing $-\epsilon_\phi$ onto $\eta \nabla_a Q$ therefore makes the model's score equal to the score of the Boltzmann policy, so that sampling from the model samples from $e^{\eta Q}$, without ever differentiating through the sampling chain and without an explicit density. The second (DDPM) term matches the score of the data distribution; the sum of the two regressions has its optimum at a weighted average of the two targets, so the learned score is that of a product of the behaviour distribution and $e^{\eta Q}$, which is a behaviour-regularised policy with α setting the trade-off.

QSM on antsoccer-arena. With the critic target of the handout (minimum of the two target critics), I could not reach the offline sanity threshold of 0.35 on this task. The whole suggested grid was run for the full 500K offline steps (seed 0):

Success at 500K (best during offline training)	η = 10	η = 30	η = 100
α = 10	0.08 (0.24)	0.00 (0.00)	0.00 (0.00)
α = 30	0.00 (0.08)	0.12 (0.20)	0.00 (0.00)
α = 100	0.00 (0.12)	0.04 (0.04)	0.08 (0.20)

The same code passes the thresholds on cube-single (0.96, threshold 0.50) and cube-double (0.44, threshold 0.10), and the loss follows the handout's equation term by term. The one difference to FQL on this task is the critic target: FQL uses the *mean* of the two target critics, which is the setting the FQL paper recommends for antsoccer because the clipped (minimum) target is too pessimistic when rewards are sparse and the horizon is long. I therefore added an option `mean_target` to the QSM agent (off by default). With the mean target, $\alpha = 30$ and $\eta = 30$ reach 0.36 at 500K steps (0.48 at best during training), which passes the threshold; $\alpha = 100$, $\eta = 10$ with the mean target stays at 0.04; with seed 1, $\alpha = 30$ and $\eta = 30$ with the mean target reach 0.44. The mean-target run is the one I use for QSM on antsoccer-arena in the rest of the report and in the submitted `exp/s2_qsm/antsoccer-arena`; QSM on the two cube tasks uses the minimum target as specified. Online, QSM collapses to 0.00 on antsoccer-arena with either target.

4.3 FQL-X compared with all baselines (Part III)

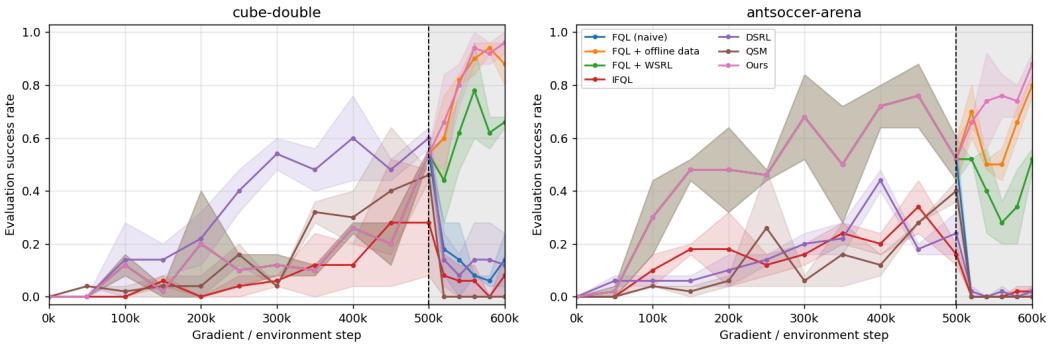


Figure 7: All methods on cube-double and antsoccer-arena. Lines are the mean of two seeds, bands the range. FQL, FQL + offline data, FQL + WSRL and FQL-X of the same seed start from the same offline checkpoint.

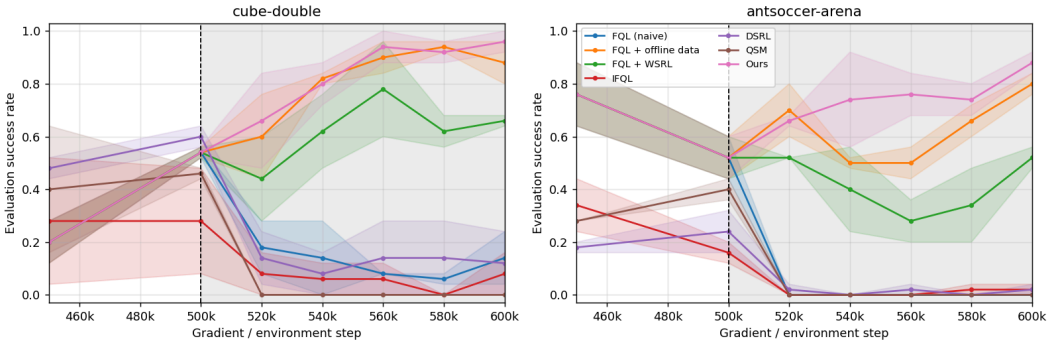


Figure 8: The same comparison, zoomed to the online phase.

Method	cube-double: offline (seeds)	final (seeds)	final mean	online mean	antsoccer-arena: offline (seeds)	final (seeds)	final mean	online mean
FQL (naive)	0.52 / 0.56	0.24 / 0.04	0.14	0.12	0.44 / 0.60	0.00 / 0.00	0.00	0.00
FQL + offline data	0.52 / 0.56	0.96 / 0.80	0.88	0.83	0.44 / 0.60	0.76 / 0.84	0.80	0.63
FQL + WSRL (20K)	0.52 / 0.56	0.64 / 0.68	0.66	0.62	0.44 / 0.60	0.48 / 0.56	0.52	0.41
IFQL	0.48 / 0.08	0.16 / 0.00	0.08	0.06	0.12 / 0.20	0.04 / 0.00	0.02	0.01
DSRL	0.56 / 0.64	0.00 / 0.24	0.12	0.12	0.16 / 0.32	0.04 / 0.00	0.02	0.01
QSM	0.44 / 0.48	0.00 / 0.00	0.00	0.00	0.36 / 0.44	0.00 / 0.00	0.00	0.00
FQL-X (ours)	0.52 / 0.56	1.00 / 0.92	**0.96**	**0.86**	0.44 / 0.60	0.92 / 0.84	**0.88**	**0.76**

"Final" is the evaluation after 500K offline and 100K online steps; "online mean" averages the five online evaluations (520K–600K) and both seeds. "FQL + offline data" uses the best amount from Section 4.2 for each task (100K on cube-double, 1M on antsoccer-arena).

Sanity check on cube-single (seed 0). Starting from the FQL checkpoint of Section 4.1 (1.00 offline), FQL-X is at 1.00 in all five online evaluations; FQL + 100K offline data and FQL + WSRL (20K) also end at 1.00 (lowest online evaluation 0.96 and 0.88).

Targets. FQL-X reaches 0.96 on cube-double (target > 0.95) and 0.88 on antsoccer-arena (target > 0.45), averaged over two seeds at the final evaluation. The cube-double result is one episode above the threshold (48 of 50 evaluation episodes), so with this evaluation size it should be read as "at the target", not as clearly above it.

Why FQL-X outperforms the baselines. The methods fall into two groups. Every method that trains on the online buffer alone (naive FQL, IFQL, DSRL, QSM) collapses on both tasks, whatever its policy class. WSRL avoids the immediate collapse but learns slowly. The two methods that keep offline data in the buffer improve steadily. This supports the analysis of Section 4.1: the failure is caused by the loss of the offline distribution in the critic's and the actor's training data, not by a weakness of the policy representation. FQL-X adds the higher update ratio to data retention. Its final success is higher than that of FQL + offline data by 0.08 on both tasks, which is within the noise of 2×25 evaluation episodes. The difference is clearer early in the online phase on antsoccer-arena, where FQL + offline data dips to 0.50 at 540K–560K steps and FQL-X stays at 0.74–0.76, and in the online mean (0.76 versus 0.63).

Ablations (seed 0, same offline checkpoint, final success at 600K).

Variant	cube-double	antsoccer-arena
FQL + 1M offline data, UTD 1 (Section 4.2)	0.88	0.76
+ UTD 4 (FQL-X): pilot run / final run	0.92 / 1.00	0.84 / 0.92
1M offline data, UTD 1, pessimistic best-of-4 ($\rho = 1$)	0.96	0.64
1M offline data, UTD 1, smaller online α (100 / 3)	0.96	0.72
Symmetric sampling 50% offline, UTD 1, N = 1	1.00	0.20
Symmetric sampling 50%, UTD 1, best-of-8	1.00	0.04
Symmetric sampling 50%, UTD 4, N = 1	1.00	0.24
Symmetric sampling 50%, UTD 4, best-of-8	0.96	0.12
Symmetric sampling 75% offline, UTD 1, N = 1	0.96	0.48

The design decisions follow from this table.

- *Cube-double is not discriminative.* Every variant that uses offline data reaches 0.92–1.00, so the choices were made on antsoccer-arena.
- *Uniform sampling from one large buffer is better than symmetric sampling.* With 50% of every minibatch from the online buffer, antsoccer-arena ends at 0.20–0.24; with 25% at 0.48; with the uniform mixture (at most 9% online data) at 0.76. Over-weighting the small online buffer reproduces a milder form of the naive collapse. On this task the online data of a half-successful policy is less informative than the offline data, so it should enter in proportion to its size.
- *Best-of-N action selection hurts on antsoccer-arena* (0.04–0.12 with N = 8, and 0.64 with pessimistic N = 4 where N = 1 gives 0.76). FQL's one-step policy already maximises Q under a distillation constraint; selecting again by Q among its samples pushes the executed actions further towards the critic's errors, and on a task with a 100-step-scale value range and a critic that is only moderately accurate, the selected actions are worse. This

is the opposite of what I expected when designing the method, and it is the reason the final configuration uses N = 1.

- *A smaller online α does not help* (0.72 on antsoccer-arena, with a slow start: 0.08 at 520K). Relaxing the constraint at the moment the critic is least reliable causes an initial drop.
- *UTD 4 is the only addition that improved antsoccer-arena* (0.76 → 0.84 and 0.92 in two runs with identical settings and seed; GPU training is not bit-wise reproducible, so the two differ). The size of this gain is comparable to the evaluation noise, so the evidence is suggestive but not conclusive; the second seed agrees in direction for the online mean.

5. Discussion

Summary. On these tasks, the single most important factor for offline-to-online fine-tuning is to keep the offline data in the training distribution. Methods that do not (four different policy classes) collapse; methods that do improve. FQL-X keeps the entire dataset, samples uniformly, and uses four updates per environment step. It reaches both targets of the project and is the best of the seven methods on both tasks in final and in mean online performance.

Limitations. (i) The method is a combination of known components (FQL, data retention and a higher UTD ratio as in RLPD); my own additions (pessimistic best-of-N selection, an online α, symmetric sampling) did not help and are reported as negative results. (ii) The improvement over FQL with retained offline data is within evaluation noise at the final step; two seeds and 25 episodes per evaluation cannot separate the two, and more seeds would be needed for a firm claim. (iii) IFQL, DSRL and QSM were run with the naive online protocol; given the results above, each of them would probably improve with retained offline data, so the comparison says more about the replay strategy than about the policy classes. (iv) QSM reaches the handout's offline threshold on antsoccer-arena only with the mean critic target, not with the specified minimum target. (v) UTD 4 makes the online phase about four times more expensive in computation, and the method requires the offline dataset at fine-tuning time, which WSRL does not.

Future work. Larger critic ensembles with random subset targets (as in REDQ and RLPD) would allow a higher UTD ratio; applying the same replay strategy to DSRL, whose latent-space exploration was the most stable naive method on cube-single, is the most direct next experiment; and evaluating with more episodes and seeds would settle whether the higher UTD ratio gives a real improvement.

Appendix: commands

```
export WANDB_MODE=offline MUJOCO_GL=egl
BASE="uv run src/scripts/train_offline_online.py --offline_training_steps 500000 --
online_training_steps 100000 --eval_interval 50000 --online_eval_interval 20000"
SINGLE=cube-single-play-singletask-task1-v0; DOUBLE=cube-double-play-singletask-task1-
v0; SOCCER=antsoccer-arena-navigate-singletask-task1-v0

# Part I
$BASE --seed=0 --run_group=s1_sacbc --base_config=sacbc --env_name=$SINGLE --alpha=300
$BASE --seed=0 --run_group=s1_fql   --base_config=fql   --env_name=$SINGLE --alpha=300
# Naive FQL on the harder tasks (also saves offline_checkpoint.pt, reused below with --
offline_checkpoint)
$BASE --seed=0 --run_group=base_fql --base_config=fql --env_name=$DOUBLE --alpha=300
$BASE --seed=0 --run_group=base_fql --base_config=fql --env_name=$SOCCER --alpha=10

# Part II (CKPT = offline_checkpoint.pt of the naive FQL run with the same task and
seed)
$BASE --seed=0 --run_group=s2_offline --base_config=fql --env_name=$DOUBLE --alpha=300
--offline_checkpoint $CKPT --offline_data {10000,100000,1000000}
$BASE --seed=0 --run_group=s2_wsrl    --base_config=fql --env_name=$DOUBLE --alpha=300
--offline_checkpoint $CKPT --wsrl_steps {1000,5000,20000}
$BASE --seed=0 --run_group=s2_ifql --base_config=ifql --env_name=$DOUBLE --expectile 0.9
$BASE --seed=0 --run_group=s2_dsrl --base_config=dsrl --env_name=$DOUBLE --noise_scale
1.0
```

```
$BASE --seed=0 --run_group=s2_qsm  --base_config=qsm  --env_name=$DOUBLE --alpha=100 --
inv_temp=10

# Part III (FQL-X)
$BASE --seed={0,1} --run_group=s3_yours --base_config=fqlx --env_name=$DOUBLE --
alpha=300 --offline_checkpoint $CKPT --config_arg num_samples=1 --offline_data 1000000
--utd 4
$BASE --seed={0,1} --run_group=s3_yours --base_config=fqlx --env_name=$SOCCER --alpha=10
--offline_checkpoint $CKPT --config_arg num_samples=1 --offline_data 1000000 --utd 4

# Figures and tables
uv run src/scripts/plot.py --runs exp_all --out report
```

A run started with `--offline_checkpoint` loads the agent after the offline phase and copies the offline part of the logs of that run, so its `eval.csv` and `train.csv` cover all 600K steps. All raw runs are in `exp_all/`; the runs selected for submission are copied to `exp/s<X>_<method>/<task>/`.